

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

A recent generation of deep-learning optimizers—Schedule-Free methods, AdEMAMix, MARS, and Lion among them—breaks with classical optimization by restructuring the momentum mechanism that lies at the core of the update rule. In a largely separate theoretical literature, provable acceleration of stochastic gradient descent (SGD) in the noise-dominated regime has been secured through a single structural device: decoupling the momentum coefficient from the weight assigned to the current gradient. This paper unifies these two hitherto disjoint developments. We derive explicit—and, in several cases, exact—correspondences establishing that each of these recently proposed optimizers is precisely an accelerated SGD method composed with preconditioning and weight averaging. We substantiate the analysis through controlled language-modeling experiments on a 150M-parameter decoder-only transformer, spanning both the small- and large-batch regimes. In the noise-dominated small-batch setting, AdEMAMix—the method that most faithfully realizes accelerated SGD—attains the strongest performance, precisely as the analysis predicts. Building directly on this correspondence, we introduce Simplified-AdEMAMix, a variant that retains the performance of AdEMAMix across both large and small batch sizes while dispensing entirely with its second momentum term. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

The past few years have produced a rapid succession of optimization algorithms for deep learning, among them Lion (Chen et al., 2023), Schedule-Free SGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these methods were advanced from distinct motivations and are conventionally presented as unrelated designs, they share a defining structural feature: each reshapes the momentum scheme that governs the optimizer update. This recurring choice, we argue, is not coincidental but symptomatic of a common underlying principle.

A largely independent line of theoretical research has pursued the acceleration of gradient descent under gradient noise. Classical momentum methods—heavy-ball and Nesterov momentum—suffice to accelerate deterministic gradient descent, most notably on quadratic objectives, yet provably fail to accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). This impossibility motivated a family of alternative momentum schemes engineered to recover acceleration in the presence of noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). A single unifying observation underlies every one of them: each proposed accelerated SGD method decouples the momentum coefficient from the weight assigned to the current gradient in the update.

The central contribution of this paper is to bridge these two research directions through a set of explicit, and in several cases exact, correspondences. We prove that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging, and we establish that Lion, Schedule-Free AdamW, MARS, and AdEMAMix are all instances of preconditioned accelerated SGD. While fragments of these relationships have surfaced in prior work (Defazio, 2021), the systematic correspondence between this recent generation of optimizers and the accelerated SGD literature has not, to our knowledge, been established before. This unified view recasts a collection of seemingly ad hoc designs as principled realizations of a single, theoretically grounded acceleration mechanism, and in doing so furnishes a common vocabulary for reasoning about when each method should be expected to help.

To validate this account, we train a 150M-parameter decoder-only transformer on 15B tokens with a small batch size of 32k tokens, placing training firmly in the noise-dominated regime in which accelerated SGD is predicted to help. Consistent with the theory, Schedule-Free AdamW tracks the accelerated SGD-based AdamW of Algorithm 3 closely, and the accelerated methods yield a measurable performance advantage at small batch sizes. Equally consistent with

the theory, this advantage recedes at sufficiently large batch sizes, where gradient noise no longer dominates—precisely the behavior that the connection to accelerated SGD anticipates.

We summarize our contributions as follows:

1. **A unifying theory.** We establish precise theoretical correspondences between accelerated SGD and the recent generation of optimizers, proving that Schedule-Free SGD, Lion, MARS, and AdEMAMix each instantiate accelerated SGD, optionally composed with preconditioning and weight averaging. This correspondence reframes disparate designs as a single mechanism and grounds every empirical prediction that follows.
2. **Empirical confirmation at small batch size.** Through controlled experiments on a 150M-parameter decoder-only transformer comparing AdamW, Schedule-Free AdamW, AdEMAMix, and MARS, we confirm the theory’s central prediction: AdEMAMix, the method that most faithfully realizes accelerated SGD, attains the strongest performance in the noise-dominated regime.
3. **Batch-size dependence.** Exactly as the equivalence to accelerated SGD predicts, the advantage of these methods over Adam vanishes at large batch sizes; consistent with the same analysis, we demonstrate that Adam equipped with a momentum schedule matches AdEMAMix in this regime.
4. **Diagnosis of Schedule-Free at scale.** At large batch sizes, Schedule-Free AdamW falls substantially behind AdamW with cosine decay, and we trace this deficit to a single concrete cause—the intrinsic coupling of the momentum and weight-averaging coefficients that Schedule-Free optimizers impose—while ruling out the ordering of preconditioning and momentum as an alternative explanation.
5. **Simplified-AdEMAMix.** Guided directly by these correspondences, we introduce Simplified-AdEMAMix, which retains a single momentum term yet matches the performance of AdEMAMix across both small and large batch-size regimes, thereby eliminating the second momentum buffer without any sacrifice in accuracy.

2 RELATED WORK

We situate our work at the confluence of two strands of the literature: the theory of accelerated SGD variants and the practical optimizers that, as we demonstrate, realize them.

The theoretical foundations were laid by Jain et al. (2018), who introduced an accelerated SGD variant with provably improved convergence rates on the least-squares problem. Kidambi et al. (2018) subsequently distilled its update rule and established rigorously that ordinary momentum affords no acceleration in this setting—the negative result that motivates decoupling the momentum coefficient from the gradient weight, and that anchors the framework we develop. A sequence of later works (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) progressively extended these guarantees to general convex and strongly convex objectives under successively weaker assumptions.

In parallel, and largely without reference to this theory, a number of practical optimizers were proposed whose update rules, we observe, echo these accelerated SGD variants even when they were not derived as such. Lucas et al. (2019) aggregated a weighted sum of several momentum terms, each with its own coefficient, into the final update; Ma & Yarats (2019) designed an optimizer explicitly inspired by the analysis of Jain et al. (2018). More recently, Chen et al. (2023) obtained an optimizer through symbolic program search that, like the earlier accelerated SGD variants, assigns distinct weights to the current gradient and to accumulated momentum, and Pagliardini et al. (2024) introduced a method that blends two momentum time scales within a single update. These recurring structural motifs are precisely the connections that we render explicit and exact in the sections that follow.

3 BACKGROUND

3.1 MOMENTUM

Momentum is the standard device for accelerating the convergence of gradient descent in the deterministic setting. For weights w_t and a momentum coefficient β , the update maintains a running accumulation of past gradients and steps along that accumulation:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a widely used variance-reduction technique in stochastic optimization: rather than returning the final iterate w_T , one reports a weighted average $\bar{w}_T$ of the optimization trajectory, formed with averaging weights γ_t , thereby attenuating the noise borne by individual iterates:

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

Throughout this paper, every instance of weight averaging employs coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We now introduce a single general framework that subsumes the accelerated SGD methods proposed across the literature under one update rule:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}$, $\alpha_{a,t}$, $\eta_{a,t}$ are possibly time-dependent scalar coefficients and g_t is the stochastic gradient evaluated at w_t ; the subscript ‘a’ marks coefficients belonging to this accelerated SGD formulation. The defining feature of the framework is that the current gradient enters the update twice—once, indirectly, through the momentum buffer m_t , and once, directly, through the term $\alpha_{a,t} g_t$ —so that the effective weight on the current gradient is decoupled from the momentum coefficient. This decoupling is exactly the ingredient that the theory identifies as necessary for stochastic acceleration.

Two observations delimit the framework’s reach. Setting $\alpha_{a,t} = 0$ collapses it to standard SGD with momentum, recovering the classical method as a special case. More consequentially, the accelerated SGD algorithms proposed across the literature—including those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023)—all fall within this single form; we give the explicit equivalences in Appendix B. The framework is therefore not one accelerated method among several but a canonical form to which the known variants reduce.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

In this section we derive precise connections between contemporary optimizers—Schedule-Free methods, Lion, MARS, and AdEMAMix—and the accelerated SGD framework of Equation (1). These derivations are not merely suggestive analogies: in each case the optimizer’s update rule is rewritten, term by term, into the accelerated SGD form, optionally composed with preconditioning or weight averaging, so that the equivalence is algebraic rather than qualitative. Building directly on the resulting insight, we then propose a simplified variant of AdEMAMix that relies on a single momentum term yet matches AdEMAMix across both the small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer designed to dispense with learning-rate scheduling altogether. Adopting the notation of Defazio et al. (2024), its update equations read:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here y_t denotes the current model weights, at which the gradient is evaluated, whereas x_t denotes the averaged weights returned for evaluation.

Our strategy is to re-express these dynamics in terms of y_t together with an auxiliary momentum variable m_t , defined by:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Expanding this definition and simplifying yields a closed recursion for m_t :

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Hence m_t obeys precisely the momentum recursion of Equation (1) under the identification $\beta_{a,t} = 1 - c_t$. With m_t in hand, we turn to the update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus y_t obeys the weight update of Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, under the identification $w_t = y_t$. The iterate at which Schedule-Free SGD computes its gradients therefore evolves exactly as accelerated SGD, with no approximation. It remains only to characterize the evaluation iterate x_t , whose dynamics we analyze next:

$$x_{t+1} = (1 - c_{t+1})x_t + c_{t+1}z_{t+1}$$

$$x_{t+1} = (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right)$$

$$x_{t+1}(1 - \beta + c_{t+1}\beta) = (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}$$

$$x_{t+1} = \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}.$$

Consequently x_t is a weighted average of the y_t trajectory; recursively expanding the recurrence shows that, for constant c_t , it is precisely an exponential average of y_t . Taken together, the two derivations establish an exact decomposition: *Schedule-Free SGD is accelerated SGD followed by weight averaging*. This equivalence, rather than any genuinely novel mechanism, is what accounts for the method's behavior, and it lets us reason about Schedule-Free SGD entirely through the accelerated SGD lens.

The decomposition attributes the effectiveness of Schedule-Free SGD to two distinct and separable ingredients:

1. improved optimization relative to standard SGD with momentum, inherited directly from its equivalence to accelerated SGD; and

2. the ability to train at a constant learning rate without a schedule, supplied by weight averaging—specifically, tailed weight averaging (see Section 4.1.3).

The particular manner in which Schedule-Free SGD/Adam realizes weight averaging confers two practical advantages:

- it requires no additional memory to store the average; and
- it exposes no explicit weight-averaging coefficient as a separate hyperparameter.

This convenience, however, is not without cost. Because a single coefficient simultaneously governs both momentum and averaging, the two cannot be tuned independently, and in Section 5.1 we show that precisely this coupling scales poorly to large batch sizes.

The two boundary values of β recover familiar algorithms as special cases of this decomposition, while the default value exposes its operating point.

4.1.1 CASE: $\beta = 0.0$

As already noted by Defazio et al. (2024), at $\beta = 0$ Schedule-Free SGD collapses to plain SGD with weight averaging; because $c_t = 1/t$, the average is taken over the entire trajectory, beginning from the very first step.

4.1.2 CASE: $\beta = 1.0$

Likewise (Defazio et al., 2024), at $\beta = 1$ the method reduces to standard momentum SGD, with the effective momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

The default configuration $\beta = 0.9$ sits between these two extremes and renders the coupling concrete:

- since c_t scales as $1/t$, the momentum coefficient grows as $1 - 1/t$;
- the ratio of the weight on the current gradient to that on the momentum is pinned at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$; and
- weight averaging effectively spans only the most recent $\approx 10\%$ of iterates.

These three consequences follow from a single shared coefficient, and it is exactly their entanglement that Section 5.1 identifies as the source of Schedule-Free AdamW’s large-batch deficit.

4.2 LION

The update rule of Lion (Chen et al., 2023) is given by:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Because this update maintains a momentum buffer while assigning separate weights β_1 and $1 - \beta_1$ to the accumulated momentum and to the current gradient, Lion (Chen et al., 2023) is exactly an accelerated SGD method composed with a coordinate-wise sign operation—the sign nonlinearity being the only element that separates it from the framework of Equation (1).

4.3 MARS

We now establish that the practical variant of MARS (Yuan et al., 2024), denoted MARS-Approx, is likewise an instance of the accelerated SGD framework composed with a preconditioning step. Ignoring bias correction and clipping, its update equations read:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t are the first- and second-moment estimates, respectively, and x_t denotes the model parameters. Introducing the reparameterized momentum $\hat{m}_t = m_t - \gamma g_t$ recasts the update in the equivalent form:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma)g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

In this form the momentum update manifestly conforms to the general accelerated SGD framework, with the current gradient entering both indirectly through $\hat{m}_t$ and directly through the γg_t term. MARS thus differs from AdamW in exactly one respect—its preconditioning is computed from the corrected gradient c_t rather than from the raw gradient—and we examine the empirical consequences of this single choice in Section 5.

4.4 ADEMAMix

Of all the methods considered here, AdEMAMix (Pagliardini et al., 2024) is structurally the closest to accelerated SGD-based AdamW. Rather than forming a linear combination of the current gradient and a single momentum term, it maintains two momentum buffers operating at different time scales and combines them linearly; the full procedure is given in Algorithm 1.

For the analysis we adopt the variant with $\beta_1 = 0$. This entails no loss of generality for the regime we study: Pagliardini et al. (2024) report that this variant nearly matches the full method at small batch sizes, and our own experiments in Section 5 corroborate this. With $\beta_1 = 0$, AdEMAMix falls squarely within the accelerated SGD framework of Equation (1). We further demonstrate that the schedules AdEMAMix prescribes for the momentum coefficient β_3 and for α —the coefficient controlling the relative weight on the current gradient—coincide closely with the schedules that the theory of accelerated SGD (Gupta et al., 2023) recommends, a correspondence that explains, rather than merely accompanies, its strong empirical performance.

In smooth convex optimization, provable stochastic acceleration requires a momentum schedule of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix reproduces this prescription closely: it scales β_3 upward over the course of training so that its momentum coefficient tracks the required $1 - k/t$ profile.

A second point of correspondence concerns the manner in which gradients accumulate. In accelerated SGD, momentum is maintained in the unnormalized form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas the momentum update in Algorithm 1 is written in EMA form:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients to m_t in accelerated SGD grows in proportion to t ; AdEMAMix reproduces exactly this growth by scaling its combination coefficient α proportionally to t . This dual alignment—in both the momentum schedule and the gradient-accumulation rate—is, we argue, precisely why AdEMAMix outperforms the competing optimizers in Figure 1.

The same analysis also predicts where AdEMAMix should struggle. At large batch sizes, Pagliardini et al. (2024) report a performance drop for the $\beta_1 = 0.0$ variant, and our framework identifies the cause unambiguously: Gupta et al. (2023) shows that the weight on the current gradient must *decrease* as the batch grows, yet AdEMAMix holds this weight fixed at 1. This persistent unit weight, rather than any incidental factor, is the mechanism behind its large-batch degradation.

This diagnosis is directly actionable. In the next section we introduce a simplified variant of AdEMAMix that (i) places an explicit, tunable weight on the current gradient, (ii) removes the need to schedule α , and (iii) retains only a single momentum term. We then verify empirically that this variant matches AdEMAMix across both the small- and large-batch setups.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

The correspondences developed above motivate a deliberately minimal design. We propose Simplified-AdEMAMix, which discards the second momentum buffer and eliminates the α schedule altogether, presented as Algorithm 2.

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

-
- 1: Sample batch B_t .
 - 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
 - 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
 - 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
 - 5: $w \leftarrow w - \eta N$
 - 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
 - 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
 - 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$
-

Its two key design choices follow directly from the accelerated SGD viewpoint: momentum is maintained in the unnormalized, theory-consistent form rather than as an EMA, and the current gradient enters the final update through a single fixed weight α —exactly the decoupled gradient weight that accelerated SGD prescribes. Setting $\alpha = 0$ recovers standard Adam under an appropriate reparameterization of η and β_1 ; the method is therefore a strict generalization of Adam rather than a departure from it. In Section 5 we show that, despite maintaining only a single momentum term, Simplified-AdEMAMix matches the full AdEMAMix across both small and large batch sizes.

5 EXPERIMENTS

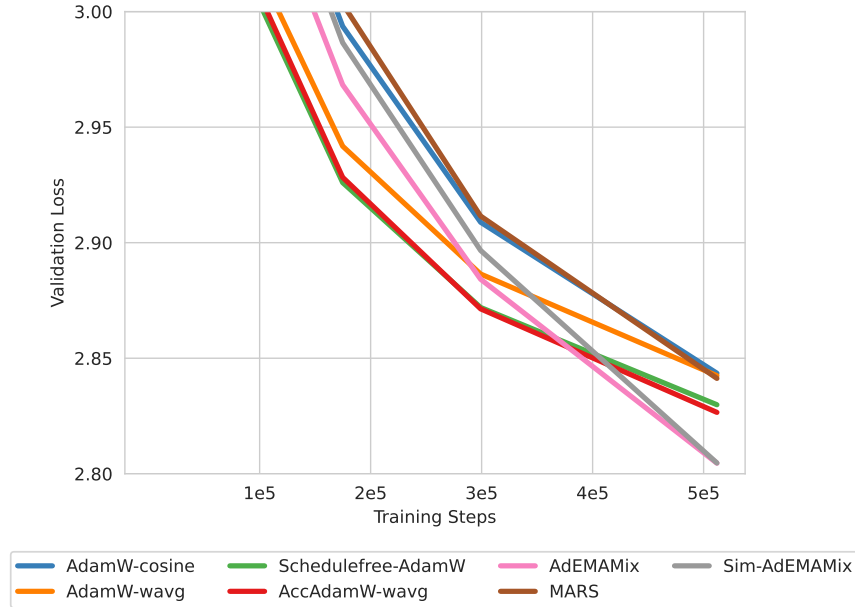


Figure 1: Small-batch (noise-dominated) regime: validation loss for the best run of each optimizer of Section 5, training a 150M-parameter decoder-only transformer on the C4 language-modeling task. AdEMAMix and Simplified-AdEMAMix attain the lowest validation loss, consistent with their being the closest realizations of accelerated SGD; Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging track one another closely, exactly as the equivalence in Equation (1) predicts, and both improve on AdamW with cosine decay.

We evaluate the theory on a language-modeling task, using a 150M-parameter decoder-only transformer trained on the C4 dataset. Training uses a sequence length of 1024 and a batch size of 32 sequences over 15 billion tokens ($\approx 5 \times$

the Chinchilla-optimal budget); the small batch places training deliberately in the noise-dominated regime, precisely the setting in which the analysis predicts that accelerated methods should help.

The comparison spans a representative cross-section of the optimizers analyzed above, together with the standard baselines:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

To ensure a fair comparison, every method is tuned over a matched hyperparameter grid; the full sweeps are reported in Appendix A.

The results in Figure 1 align cleanly with the theory on every point. First, Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging achieve comparable validation loss—the direct empirical signature of the equivalence proven in Equation (1). Second, both surpass AdamW with cosine decay as well as AdamW with tailed weight averaging, which establishes that the accelerated momentum, and not the averaging alone, is what drives the gain. Third, AdEMAMix and Simplified-AdEMAMix attain the lowest validation loss among all methods considered; because these two are the closest realizations of accelerated SGD, this ordering is exactly what the analysis anticipates. Notably, Simplified-AdEMAMix matches full AdEMAMix here despite carrying only a single momentum term.

5.1 LARGE BATCH SIZE EXPERIMENTS

The theory predicts not only where accelerated methods help but also, symmetrically, where their advantage should vanish. Whereas the small-batch experiments above operate under noisy gradients, we now probe the complementary large-batch regime, training with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale). Because acceleration is fundamentally a property of the noise-dominated regime, the connection to accelerated SGD predicts that the small-batch gains should recede here—and the experiments bear this out, while additionally exposing a specific and diagnosable failure of Schedule-Free AdamW.

Schedule-Free AdamW. Figure 2 shows Schedule-Free AdamW falling well behind AdamW at large batch size. This gap is a direct consequence of the momentum–averaging coupling identified in Section 4.1.3: at large batch sizes the optimal momentum lies well below $1 - 1/t$, and although one may substitute a faster decay $\approx 1 - r/t$ with $r \geq 1$, increasing r simultaneously shrinks the effective weight-averaging window, so that no single coefficient can discharge both roles at once. We further rule out the natural alternative explanation. Schedule-Free AdamW and AdamW differ in the order of momentum and preconditioning—AdamW applies momentum first, whereas Schedule-Free AdamW preconditions first, in the manner of LProp (Ziyin et al., 2021). Yet Figure 2 shows LProp performing on par with AdamW, which isolates the coupling, and not the ordering, as the cause of the deficit.

AdEMAMix. Consistent with Pagliardini et al. (2024), Figure 3 shows that the $\beta_1 = 0.0$ variant of AdEMAMix drops sharply relative to the two-momentum version at large batch size. Our analysis names the mechanism precisely: AdEMAMix pins the weight on the current gradient at 1, whereas the accelerated SGD theory of Gupta et al. (2023) requires this weight to diminish as the batch grows. Simplified-AdEMAMix is designed expressly to restore that missing degree of freedom, and Figure 3 confirms that it recovers the full AdEMAMix performance while carrying only one momentum term. Strikingly, it does so at $\alpha = 0.0$, so that in this regime Simplified-AdEMAMix reduces to standard Adam with a momentum schedule—a direct demonstration that Adam, equipped with an appropriate momentum schedule, already matches AdEMAMix at large batch sizes.

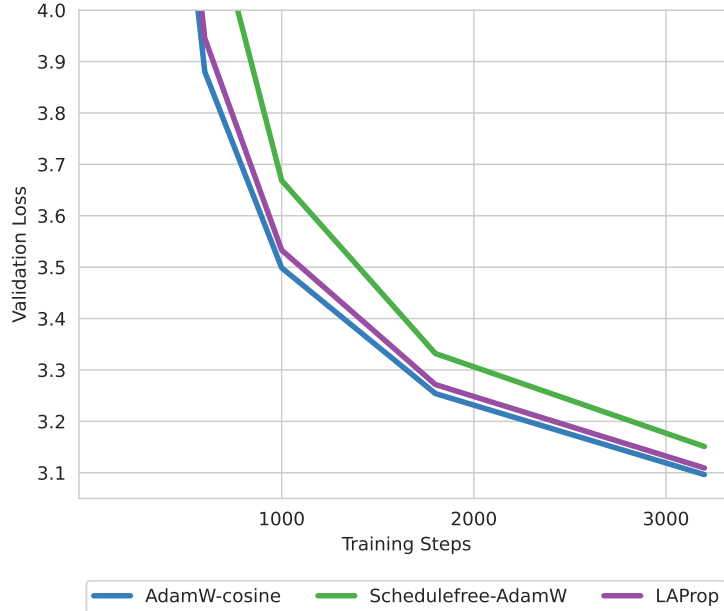


Figure 2: Large-batch regime: validation loss for the best runs of AdamW with cosine decay, Schedule-Free AdamW, and LARProp at a batch size of 1M tokens. Schedule-Free AdamW trails AdamW, whereas LARProp tracks AdamW closely—an outcome that isolates the momentum–averaging coupling, rather than the order of preconditioning and momentum, as the source of the gap. Experimental details are given in Section 5.1.

6 CONCLUSION

This work draws a collection of recently proposed optimizers—Schedule-Free methods, AdEMAMix, MARS, and Lion—into a single, theoretically grounded picture, establishing that each is an instance of accelerated SGD, optionally composed with preconditioning and weight averaging. The unifying account is far more than descriptive: it explains our empirical finding that AdEMAMix, the closest realization of accelerated SGD, achieves the strongest performance in the noise-dominated small-batch regime, and it correctly predicts both the disappearance of that advantage at large batch sizes and the specific large-batch failure of Schedule-Free AdamW.

The same viewpoint yields a concrete algorithmic payoff. Guided directly by the correspondence, we introduce Simplified-AdEMAMix, which retains a single momentum buffer and dispenses with the α schedule, yet matches full AdEMAMix across both small and large batch sizes—eliminating the second-momentum memory overhead at no cost in performance. More broadly, reading these optimizers through the lens of accelerated SGD converts a set of independently motivated heuristics into principled design choices, and supplies a coherent basis for reasoning about when each method should be expected to help.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.

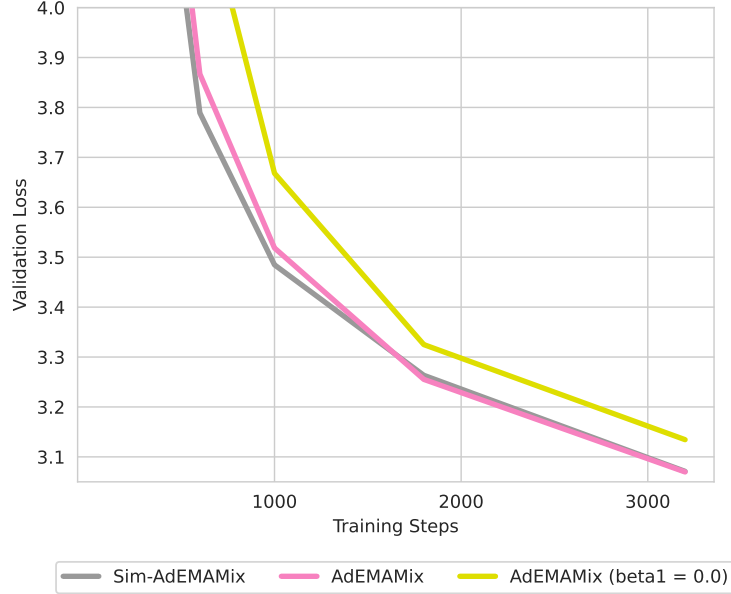


Figure 3: Large-batch regime: validation loss for AdEMAMix with and without $\beta_1 = 0.0$, alongside Simplified-AdEMAMix, at a batch size of 1M tokens. The $\beta_1 = 0.0$ variant degrades, exactly as predicted by its fixed unit weight on the current gradient, whereas Simplified-AdEMAMix recovers the full AdEMAMix performance with only a single momentum term. Experimental details are given in Section 5.1.

Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.

Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.

Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.

Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.

Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.

Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.

Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=S1fUpoR5FQ>.

Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.

- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

To ensure a fair comparison, each optimizer was tuned over its own grid of the hyperparameters most relevant to it. We report the grids for the small-batch experiments first, and the corresponding large-batch grids thereafter.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values were $\beta_1 = .9$ and $\beta_2 = .999$, coinciding exactly with the defaults. Whereas $\beta_2 = .95$ is the common choice at larger batch sizes, the advantage of a higher β_2 in the small-batch regime is consistent with the observations of Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, i.e. β_1, β_2 fixed to the optimal values from the previous sweep. This shorter warmup performed worse than the 51.2k-step warmup.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The corresponding sweeps for the large-batch experiments are as follows.

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

This appendix substantiates the claim made in the main text: every accelerated SGD method of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) is a special case of the general form in Equation (1). For each method, we algebraically rewrite the original update into that form, thereby exhibiting the decoupling of the momentum coefficient from the gradient weight explicitly and confirming that the framework is genuinely canonical.

B.1 AGNES

The update of Gupta et al. (2023) is given by:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n and the objective is evaluated at x_n . Eliminating v_n recasts these updates as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

so that x'_{n+1} obeys precisely the update of Equation (1).

B.2 ASGD

The update of Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} and the objective is evaluated at the tail-averaged iterate x . These updates can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

which again conform to the form of Equation (1).

B.3 MASS

The update of Liu & Belkin (2020) is given by:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t and the objective is evaluated at w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

which likewise conform to the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update of Vaswani et al. (2019) is given by:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$

which are, once more, of the accelerated SGD form of Equation (1), thereby completing the equivalence for all four methods.